# Coding / Final Architecture

PDF edition of preserved repository document | docs/coding/final_architecture.md

Historical statements and test counts below retain their original scope. Consult the master report for current status.

## Coding research architecture

The historical integer multiplication environment and release remain intact. Coding is a separate adapter over the existing Goal, Entity, Relation, benchmark and Prediction interfaces. Its task/state/action/event/result/transition/trajectory records represent actual repositories and tool calls, rather than arithmetic-specific operations.

A learner receives the broken repository, specification and visible tests. Private test inputs and expected values, reference sources and expert patches remain in dataset/evaluator records outside its observation. Training-only expert trajectories are actual executed inspections, visible failures, accepted repairs and visible verification; hypothetical plans are excluded from verified experience memory. The benchmark wrapper constructs a learner task without private cases and delegates terminal assessment to an independent backend.

The thirteen typed coding tools inspect repositories, files, symbols and visible tests, search, edit or apply a unique patch, run one/all visible tests, inspect errors, revert, compare versions and finish. Repository paths reject absolute/traversal names. Real test execution uses a pinned CPython 3.14.7 WASI guest, with capability-limited filesystem mounts, resource fuel, bounded output, memory and wall time. No model-generated Python runs in the host interpreter. Host-side expected-value comparison requires matching value and type, not merely process exit zero.

The primary backbone is Qwen2.5-Coder-1.5B-Instruct revision 2e1fd397ee46e1388853d2af2c993145b0f1098a. It runs locally in float32 on CPU. Rank-eight q_proj/v_proj LoRA trains 1,089,536 parameters. Four conditions share this backbone and nested unique training-task budgets: plain repair, structured repair, persistent typed-action policy and actual-experience replay. Supervision, input tokens, runtime feedback and interaction budgets differ; this is not an architecture-only causal comparison. Replay updates policy weights using accepted repair actions and real past feedback. It is offline expert replay, not reinforcement learning or self-play. Inspection/testing/stopping actions rely on the pretrained model: accepted repair is the only trained action type.

Working memory resets per episode. Verified episodic memory contains only executed training examples; specification-word retrieval and concept records are bounded. Evaluation repairs never become verified training experience. The relation graph records files, symbols and calls. The planner compares two bounded branches using static preconditions/syntax. Predicted states are marked hypothetical, and predicted test success is null. It is not a learned world model; actual execution alone can establish success.

The secondary SQL environment reuses Goal/Entity/Relation and Model/Prediction records with typed schema/table inspection, query editing, execution and stopping. Its in-memory SQLite authorizer permits SELECT capabilities, with an execution deadline, output limits and no extension/file access. Private database rows and expected queries are terminal-only. The small SQL study tests interface portability without SQL fine-tuning.